

Assignment 3

AIL7002

[Starter Code](#)

October 5, 2026

Part 1 — Sorting and IntroSort

1 Objective

In this part of the assignment, your task is to implement a collection of sorting components and combine them into an **IntroSort** implementation. The assignment is intended to evaluate your understanding of sorting algorithms, asymptotic complexity, memory management, recursion, and the design of hybrid algorithms.

You are required to implement only the sorting techniques that are directly used by the final IntroSort implementation: **Insertion Sort, Quick Sort, and Heap Sort**. The other sorting techniques discussed in class are not required to be implemented for this assignment.

The final implementation must remain correct on both ordinary and adversarial inputs. In particular, your implementation will be tested on random, sorted, reverse-sorted, duplicate-heavy, and pivot-hostile inputs. Correctness alone is therefore insufficient; the implementation must also satisfy the stated complexity and memory requirements.

2 Requirements and Restrictions

The following requirements apply to the entire submission.

1. Language: **C++17 only**.
2. Implement the required ADT from scratch. STL containers are not part of the solution.
3. **Do not use** `std::sort`, `std::stable_sort`, `qsort`, or the STL heap helpers `std::make_heap`, `std::push_heap`, and `std::pop_heap`.
4. Do not use STL sequence or associative containers such as `std::vector`, `std::deque`, `std::list`, `std::map`, `std::set`, `std::unordered_map`, or `std::unordered_set`.
5. Lightweight utilities such as `iostream`, `chrono`, `random`, `cstdint`, `cstdintdef`, `limits`, and `utility` are permitted. Algorithmic data structures and sorting routines must be implemented by you.
6. The production Quick Sort implementation must use a bounded-stack strategy. An implementation that can recurse to depth $\Theta(n)$ on an adversarial input is considered incorrect, even if it eventually produces a sorted array.
7. Avoid signed-integer overflow. In particular, range lengths, child indices, and midpoint calculations must not rely on unchecked signed arithmetic.
8. Memory leaks, use-after-free, and undefined behavior are considered correctness failures.

Adversarial inputs. Hidden tests will include duplicate-heavy inputs, already-sorted arrays, reverse-sorted arrays, all-equal arrays, small partitions, odd input sizes, and extreme signed 64-bit keys such as `INT64_MIN` and `INT64_MAX`.

3 Dynamic Array ADT

Implement `DynamicArray<T>` in `include/core/dynamic_array.hpp` using manually managed dynamic storage. The implementation must correctly manage ownership and support both copy and move semantics.

The following operations are required:

- `size()`, `capacity()`, and `empty()`;
- `operator[]` and a checked `at()`;
- `reserve()`, `push_back()`, `pop_back()`, and `clear()`;
- copy constructor and copy assignment operator;
- move constructor and move assignment operator;
- destructor with no memory leaks.

Repeated append operations must have amortized $O(1)$ complexity. The implementation should therefore increase its capacity geometrically rather than reallocating on every insertion.

The sorting algorithms may operate directly on raw pointers where appropriate. The purpose of this ADT component is to assess correct dynamic-memory ownership and implementation of a general-purpose contiguous storage abstraction; it is not necessary to force the ADT into every sorting routine.

4 Required Sorting Algorithms

You are required to implement the following three sorting algorithms.

Algorithm	Required properties	Target complexity
Insertion Sort	Stable and in-place; use the standard shifting implementation.	Worst-case $O(n^2)$
Quick Sort	In-place with robust partitioning and bounded stack usage.	Expected $O(n \log n)$
Heap Sort	In-place binary heap implemented from scratch.	Worst-case $O(n \log n)$

Instrumentation. Use the provided `SortStats` structure. Increment `comparisons` when the algorithm performs a key comparison and update `moves` and `swaps` consistently when elements are moved. The counters are intended for analysis and diagnostics and do not replace the correctness requirements.

4.1 Insertion Sort

Implement the standard in-place insertion sort. The algorithm must be stable, and its worst-case running time should be $O(n^2)$.

4.2 Quick Sort

Your Quick Sort implementation must satisfy all of the following requirements:

1. The pivot-selection strategy must not always select a fixed endpoint.
2. The partitioning procedure must handle inputs containing many equal keys without repeatedly producing highly unbalanced partitions.
3. The recursion strategy must ensure bounded stack usage. One acceptable approach is to recurse on the smaller partition first and process the larger partition iteratively.
4. The implementation must expose a partitioning helper that can be reused by IntroSort.

A median-of-three pivot-selection strategy combined with three-way partitioning is one acceptable implementation. Alternative strategies are permitted provided that their correctness and complexity properties are clearly justified in the written analysis.

4.3 Heap Sort

Implement Heap Sort using a binary heap represented within the input array. The heap operations, including heap construction and the sift-down operation, must be implemented from scratch. The final algorithm must sort the input in-place with worst-case $O(n \log n)$ running time.

5 IntroSort

The three algorithms above must now be combined into a single hybrid sorting algorithm, **IntroSort**. IntroSort begins with Quick Sort, uses a depth limit to detect repeated unfavorable partitioning, switches to Heap Sort when the depth budget is exhausted, and uses Insertion Sort for sufficiently small partitions.

A standard high-level structure is shown below.

```
intro_sort(a, n):  
    depth_limit = 2 * floor(log2(n))  
    introsort_loop(a, n, depth_limit)  
    insertion_sort(a, n)
```

An alternative implementation may sort sufficiently small partitions immediately using Insertion Sort and then perform a final insertion-sort pass. Either approach is acceptable provided that the final implementation satisfies the stated correctness and complexity requirements.

5.1 Required IntroSort Policy

The final sorting pipeline must use all three required sorting algorithms:

- **Insertion Sort cutoff:** choose a small threshold, typically in the range 16–32 elements, and justify the selected value;
- **Quick Sort phase:** use the Quick Sort partitioning mechanism implemented in the previous section;
- **Heap Sort fallback:** when the recursion-depth budget is exhausted, sort the corresponding active subrange using Heap Sort.

5.2 Depth Limit

The depth limit must be proportional to $\log n$. A standard choice is

$$2 \lfloor \log_2 n \rfloor.$$

The objective is to prevent a sequence of unfavorable Quick Sort partitions from producing $\Theta(n)$ recursion depth. Once the specified budget is exhausted, the implementation must use Heap Sort on the remaining active subrange.

The constant factor in the depth limit may differ from the standard value if the choice is justified. The important requirement is that the depth limit is logarithmic in the input size and that the Heap Sort fallback is actually integrated into the IntroSort control flow.

5.3 Correctness Invariants

At every stage of the IntroSort implementation, the following conditions must hold:

- active subrange boundaries are valid and do not overlap;
- elements outside the active subrange have already been placed in their appropriate final region;
- the remaining depth budget never increases;
- the sorting routine invoked as a fallback receives exactly the subrange for which it is responsible.

The implementation should be sufficiently modular that the Quick Sort partitioning routine and Heap Sort fallback can be tested independently.

6 Benchmarking and Adversarial Test Cases

Include a benchmark driver that generates and evaluates at least the following input families:

- random values;
- sorted arrays;
- reverse-sorted arrays;
- arrays that are approximately 90% sorted with a small number of local disruptions;
- arrays containing only one distinct value;
- arrays drawn from a small value domain and therefore containing many duplicates;
- a pivot-hostile input pattern of your choice;
- inputs containing extreme signed 64-bit values.

For each input family, report the input size, execution time, and the relevant statistics collected by `SortStats`. A simple command-line benchmark driver is sufficient; a separate benchmarking framework is not required.

7 Required File Interface

Unless a private helper is required, use the following public names and interfaces in the starter repository.

```
// include/sortlab_types.hpp
namespace sortlab {
using Key = std::int64_t;
struct SortStats {
    std::uint64_t comparisons = 0;
    std::uint64_t moves = 0;
    std::uint64_t swaps = 0;
    std::uint64_t passes = 0;
};
}

// include/sort/sort_kernels.hpp
void insertion_sort(Key* a, std::size_t n, SortStats* stats = nullptr);
void quick_sort(Key* a, std::size_t n, SortStats* stats = nullptr);
void heap_sort(Key* a, std::size_t n, SortStats* stats = nullptr);
void intro_sort(Key* a, std::size_t n, SortStats* stats = nullptr);
```

8 Grading Rubric

Evaluation criteria. In addition to functional correctness, submissions will be evaluated on the quality of the decomposition, correctness of memory management, adherence to the required complexity bounds, bounded recursion behavior, appropriate use of the IntroSort fallback, and the quality of the submitted analysis and benchmark results.

The objective is to implement the components required for a robust hybrid sorting algorithm and to understand the design decisions that allow IntroSort to combine the practical advantages of Quick Sort with worst-case protection from Heap Sort and efficient handling of small partitions by Insertion Sort.

Part 2

1 Motivation

In this assignment your task is to **migrate a database from one device to another** and then **serve queries on the new device**. Both devices are characterized by a small memory model described in the next section. The database is a key-value store, implemented as a B+ tree.

When the database is migrated from one device to another, the design of the B+ tree must be changed to accommodate the memory model of the new device. In particular the branching factor and fill factor of the tree must be adjusted.

Each read and write operation from the disk costs a certain amount, and your goal is to migrate the database by incurring as small a cost as possible.

2 Storage Model

2.1 Memory model tuple

The data is stored on a simulated “disk”, the disk is organized into pages. A page is the smallest unit that is retrieved by a read operation on this disk. Once a page has been read it is stored in a buffer. While the page remains in the buffer, the write and reads from that page incur no cost.

Every device is thus described by the tuple

$$\mathcal{M} = (R, W, P, M)$$

- R : the cost of reading one page.
- W : the cost of writing one page.
- P : the page size.
- M : the capacity of the device’s buffer, measured in pages.

There are two devices: the *old* device $\mathcal{M}_{\text{old}} = (R_{\text{old}}, W_{\text{old}}, P_{\text{old}}, M_{\text{old}})$ that holds the source database, and the *new* device $\mathcal{M}_{\text{new}} = (R_{\text{new}}, W_{\text{new}}, P_{\text{new}}, M_{\text{new}})$ that you migrate to. **Note that write cost of old device could be much higher than other costs.**

2.2 Pages

A page is an array of exactly P signed 64-bit integers (*words*). Keys, values, page identifiers, counters and flags are all words. A page therefore holds at most P words, and every layout decision you make must fit inside it. The device rejects any page whose length is not P .

2.3 Disk API

Both devices expose the same interface (defined in `disk.h`, which you must not modify).

```
using PageId = int64_t; // similar to long long
using Page    = std::vector<int64_t>; // always P words

struct MemoryModel { double read_cost, write_cost; size_t page_words, buffer_pages; };

class Disk {
public:
    const MemoryModel& model() const;
    Page read_disk(PageId id); // cost R unless the page is in the
    // buffer
    void write_disk(PageId id, const Page&); // always cost W
```

```

PageId allocate_page();           // free of charge, contents undefined
void free_page(PageId id);       // free of charge
};

```

Buffer semantics. Each device has its own buffer of M pages, managed automatically by the device.

- `read_disk(id)` on a page that is in the buffer is free. On a page that is not in the buffer it costs R , and the page is inserted into the buffer.
- If the buffer would then exceed M pages, the page that was *inserted earliest* is evicted (FIFO). A buffer hit does *not* refresh a page's position: re-reading a page that is already in the buffer does not delay its eviction.
- `write_disk(id, page)` always costs W . If the page is currently in the buffer, its buffered copy is updated in place and keeps its position. If it is not in the buffer, it is not inserted only the disk copy is modified.
- `read_disk` returns a copy. Modifying the returned page changes nothing until you write it back.
- The buffer starts empty at the beginning of each phase (Section 5).

Page identity and layout. `allocate_page` returns a fresh page identifier. You may write to any allocated page in any order. Placing related pages at consecutive identifiers (a clustered layout) or scattering them (unclustered) is entirely your choice. *The cost model does not distinguish the two*: only the number of reads and writes counts.

2.4 Cost of a run

The device counts every buffer-missing read and every write. Let r_o, w_o, r_n, w_n be the counts of reads and writes on the old and new device during a phase. The cost of the phase is

$$C = R_{\text{old}} r_o + W_{\text{old}} w_o + R_{\text{new}} r_n + W_{\text{new}} w_n$$

Buffer hits and in-memory computation are free. The cost is reported to you only after a whole test case finishes.

3 The Database

The database is a set of records (k, v) with distinct 64-bit integer keys and 64-bit integer values, stored on the old device as a B⁺-tree. Its layout is fixed and public.

Page kind	Layout (words)	Capacity m for page size P
Leaf	[1, count, next, $k_1, v_1 \dots k_m, v_m$]	$m = \lfloor (P - 3)/2 \rfloor$ records
Internal	[0, count, $c_0, k_1, c_1 \dots k_m, c_m$]	$m = \lfloor (P - 3)/2 \rfloor$ keys, $m + 1$ children

Leaves are chained in key order through `next_leaf` (-1 at the end). The old tree was built by a long history of inserts and deletes, so its leaves are *not* uniformly full. You do not have to parse this layout yourself: the provided `LegacyScanner` (in `legacy.h`) walks it and yields all records in ascending key order through the `RecordSource` interface, paying the reads on the old device as it goes. You are free to bypass it and read the old device directly if you find a cheaper way to do so.

The new tree has **your own layout**. The only fixed requirement is that page 0 of the new device is preallocated for you as the *meta page*, and after a restart (Section 5) your code must be able to recover the whole tree from the new device alone.

4 What You Implement

You edit only `bptree.h` and `bptree.cpp`. Everything else is provided. Your work has three parts: (1) a *design function* that decides the shape of the tree, (2) the *tree operations* themselves, and (3) a *page layout* of your own choosing that these operations read and write through the disk API. The listings below show what is already declared for you (the types in Listings 1 and 2 are provided) and what you must fill in (Listings 3 and 4).

4.1 Listing 1: Basic types (provided)

These are the types you use in your code. These wrap primitive types so that you don't have to think of the underlying datatype and can reason in the memory level abstraction defined in Section 2. A `Record` is one key–value pair.

- A `Record` is one key–value pair.
- A `RecordSource` is a one-pass, ascending-order stream of records; the migration hands you one that reads the old database (Section 3), and `bulk_fill` consumes it.
- `WorkloadHint` summarises what the query phase will do so that you can tune the tree for it. It only describes the workload in aggregate; you never see the actual operations in advance.

It only describes the workload in aggregate; you never see the actual operations in advance.

```
using Key = int64_t; using Value = int64_t;
struct Record { Key key; Value value; };

struct RecordSource {
    virtual bool next(Record& out) = 0;           // sorted stream of records
                                                    // false when exhausted
};

struct WorkloadHint {
    double p_find, p_range, p_insert, p_delete;   // operation mix, sums to 1
    double mean_range_length;                     // records per range query
    enum class InsertPattern { Uniform, Ascending, Hotspot } insert_pattern;
    size_t expected_records;                       // N at migration time
};
```

4.2 Listing 2: Tree configuration (provided type, you fill in the values)

`TreeConfig` holds the four numbers that define your tree's shape: the branching factor (capacity of a leaf and fan-out of an internal node) and the fill factors used when bulk loading. Capacities are hard limits used by `insert` and `remove`. Fill factors are only used by `bulk_fill`: a fill of 0.7 means each node is packed to 70% of its capacity, leaving room so that later inserts do not immediately split it. The configuration is saved in the meta page so it survives the restart.

```
struct TreeConfig {
    size_t leaf_capacity;           // L: max records per leaf, L >= 2; a full leaf must fit in
    size_t internal_fanout;         // F: max children per internal node, F >= 3; must fit in a
    double leaf_fill;              // f_L in [0.5, 1]; bulk-load target ceil(f_L * L) records
    double internal_fill;          // f_I in [0.5, 1]; bulk-load target ceil(f_I * F) children
};
```

4.3 Listing 3: Design function (you implement)

This single function is where you decide the branching factor and the fill factors. It is called once, before the migration, and receives both memory models and the workload hint. Values that exceed what one page can hold are rejected by the harness. This is the part of the assignment where the cost model matters most.

```
TreeConfig choose_config(const MemoryModel& old_model,
                        const MemoryModel& new_model,
                        const WorkloadHint& hint);
```

4.4 Listing 4: The B⁺-tree class (you implement)

This is the tree. `create` starts an empty tree on the new device and `open` reattaches to an existing one after the restart, so the two together define what lives on disk. The remaining methods are the five operations of the assignment: `bulk_fill`, `insert`, `remove`, `find` and `range_find`, plus `sync`, which the harness calls at the end of each phase. The comment beside each method states its contract.

```
class BPlusTree {
public:
    static BPlusTree create(Disk& disk, const TreeConfig& config); // empty tree, writes
        meta page
    static BPlusTree open(Disk& disk); // recovers a tree
        from disk alone

    void bulk_fill(RecordSource& sorted_records); // empty tree only; ascending unique
        keys
    void insert(Key key, Value value); // inserts, or overwrites the value of
        an existing key
    bool remove(Key key); // false if the key was absent
    std::optional<Value> find(Key key);
    std::vector<Record> range_find(Key low, Key high); // all records with low <= key <=
        high, ascending
    void sync(); // make the on-disk state complete and
        current
};
```

(`remove` plays the role of `delete`, which is a reserved word in C++.) The semantics in detail:

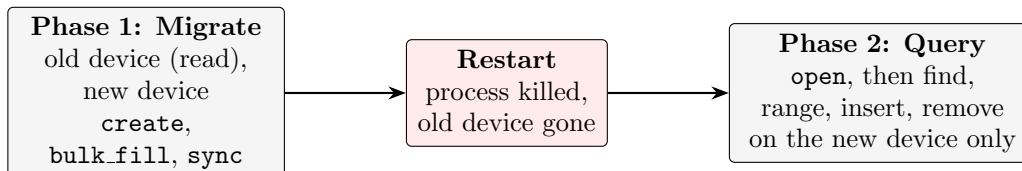
- `bulk_fill` builds the tree bottom-up from a sorted stream in a single pass, packing leaves and internal nodes to the configured fill factors. It may be called only on an empty tree.
- `insert` and `remove` must keep the tree a valid B⁺-tree after every operation. Let $L = \text{leaf_capacity}$ and $F = \text{internal_fanout}$.
 - All leaves are at equal depth.
 - Every non-root leaf holds between $\lceil L/2 \rceil$ and L records.
 - Every non-root internal node has between $\lceil F/2 \rceil$ and F children, i.e. between $\lceil F/2 \rceil - 1$ and $F - 1$ keys.
 - A root leaf holds between 0 and L records. A root internal node has between 2 and F children.
- `range_find` must use the leaf chain after locating the first leaf; it must not re-descend from the root for each leaf.
- `sync` is called by the harness at the end of every phase. After it returns, the new device must contain everything needed for `open` to reconstruct the tree. Any write it performs is charged.
- `open` is called at the start of Phase 2, in a fresh process. It reads the meta page (one page read), decodes the root page id, the height and the `TreeConfig`, and returns a `BPlusTree`

built from them. It must not read or rebuild the rest of the tree. This is needed as `BPlusTree` object is not persisted after Phase 1. The writes made to disk persist and thus all pages are present, only identify the root, height and config to build the `BPlusTree` object again.

- **The meta page** is page 0 of the new device, preallocated for you and blank until you write it. You choose its layout and you write it: use `create` to write it. What `open` decodes must match exactly what you wrote.

5 Phases and the Restart

Every test case runs in two phases, in two *separate processes*.



Phase 1 (migration). The harness gives your process both devices. You call `choose_config`, `create`, `bulk_fill`, and the harness calls `sync`. Cost:

$$C_{\text{mig}} = R_{\text{old}}r_o + W_{\text{old}}w_o + R_{\text{new}}r_n + W_{\text{new}}w_n.$$

Restart. The process is terminated and a fresh one is started. Nothing in memory survives; only the contents of the new device do. The old device is no longer accessible, and the new device's buffer is empty.

Phase 2 (queries). The fresh process calls `BPlusTree::open`, then a fixed, seeded sequence of operations is executed against your tree, then `sync`. Every answer is compared with a reference. Cost:

$$C_{\text{query}} = R_{\text{new}}r_n + W_{\text{new}}w_n.$$

Your final cost for a case is $C_{\text{mig}} + C_{\text{query}}$, reported per phase, together with the raw counts r_o, w_o, r_n, w_n .

6 Provided Files and the Grader

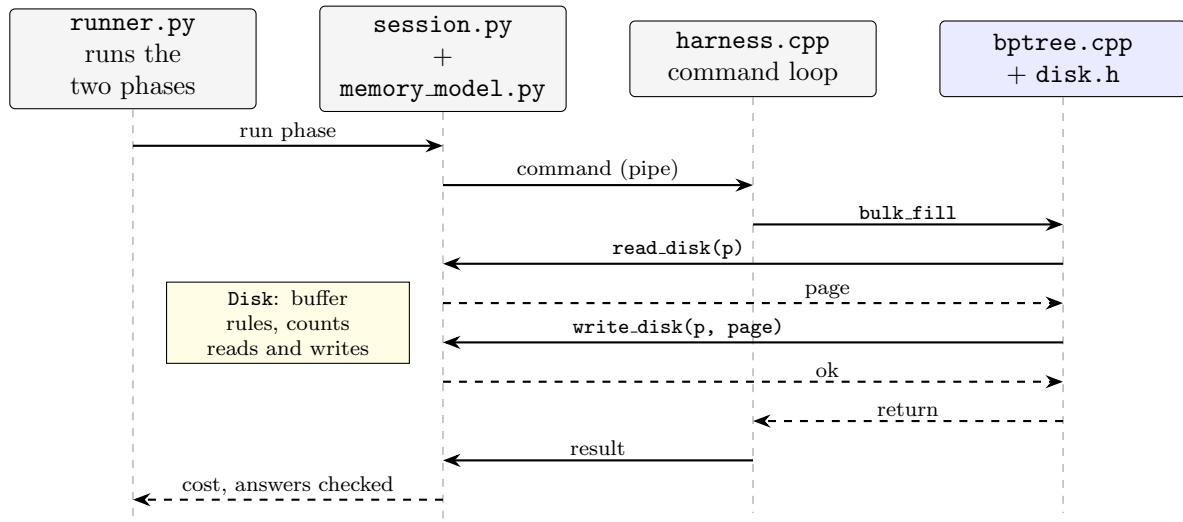
6.1 Files provided

You edit only `bptree.h` and `bptree.cpp`. Everything else is provided and must not be modified.

File	Role
<code>bptree.h</code> , <code>bptree.cpp</code>	Your solution (stubs provided)
<code>types.h</code>	<code>Record</code> , <code>RecordSource</code> , <code>WorkloadHint</code> , <code>TreeConfig</code>
<code>disk.h</code>	The disk API: <code>read_disk</code> , <code>write_disk</code> , <code>allocate_page</code> , <code>free_page</code>
<code>legacy.h</code>	<code>LegacyScanner</code> , a <code>RecordSource</code> over the old database
<code>harness.cpp</code> , <code>wire.h</code> , <code>wire.cpp</code>	The command loop that calls your code, and the pipe framing
<code>Makefile</code>	<code>make</code> builds <code>solution</code> ; <code>make functional</code> and <code>make public</code> run the cases
<code>grader/</code>	The Python grader: <code>runner.py</code> , <code>session.py</code> , <code>memory_model.py</code> , <code>legacy_tree.py</code> , <code>workload.py</code> , <code>cases.py</code>
<code>cases/</code>	Public test cases (JSON)

6.2 How the grader and your code interact

The grader (Python) starts your executable as a child process and drives it one command at a time. Every disk call your code makes travels over a pipe to the grader, which applies the buffer rules and charges the cost. The diagram follows one migration command.



Only `bptree.cpp` (shaded) is yours. Between phases the grader kills the process and starts a fresh one, so nothing survives except what is on the new device.

6.3 Example test case

A test case is a small JSON file. This is `cases/case1.json`:

```
{ "name": "case1",
  "record_count": 200000,
  "old": { "read_cost": 8, "write_cost": 64, "page_words": 64, "buffer_pages": 4 },
  "new": { "read_cost": 1, "write_cost": 1, "page_words": 256, "buffer_pages": 8 },
  "workload": { "p_find": 0.4, "p_range": 0.1, "p_insert": 0.3, "p_delete": 0.2,
    "mean_range_length": 50, "insert_pattern": "uniform" },
  "operation_count": 20000,
  "seed": 1 }
```

From the seed the grader generates 200 000 random records and stores them on the old device as a B⁺-tree with 64-word pages. It then generates 20 000 operations with the given mix and their correct answers. Your `choose_config` receives the `old` and `new` models and the `workload` fields as its `WorkloadHint`; it never sees the operations themselves.

7 Rules

The whole point is to reason about pages, so the rules protect the model.

- R1.** All persistent data goes through `read_disk` and `write_disk`. No files, memory-mapped regions, sockets, threads or shared memory.
- R2.** You may hold in memory, at any moment, only the pages needed by the operation in progress (for example the root-to-leaf path of a split), plus a small constant number of scalars such as the root identifier and the height. You must not keep a cache of pages across operations, and you must not hold the database, or any sizeable part of it, in a `map`, `vector`, or any other container. The grader enforces this by measuring the growth of your process's peak resident memory during each phase; a growth beyond the case's allowance (2 MiB by default, far smaller than the database) fails the case. Code review will also check this.

- R3.** Because of the restart, state kept in members that you have not written to disk is lost. Persist what you need in the meta page or elsewhere on the new device, and account for those writes: a design that rewrites the meta page on every insert pays for it.
- R4.** Do not modify `disk.h`, `legacy.h`, or the harness. Do not attempt to inspect the harness's counters or the device's internals.
- R5.** Correctness is required in every test. A wrong answer in Phase 2 zeroes the cost score of that case.
- R6.** The provided `legacy.h` may be used freely, and you may write additional helper files that you include from `bptree.cpp`, provided the rules above hold.

Public cases. The following are provided to test your work. Hidden cases will vary all parameters.

Case	N	R_{old}	W_{old}	P_{old}	R_{new}	W_{new}	P_{new}	M_{new}	Workload
1	2×10^5	8	8	64	1	1	256	8	mixed, uniform
2	5×10^5	1	1	128	2	10	512	16	write-heavy, hotspot
3	3×10^5	5	5	256	5	5	32	4	read-heavy, long range
4	10^6	2	20	128	4	4	128	32	append-only

8 Submission and Running

Submit `bptree.h`, `bptree.cpp`, any extra helper files, and `report.pdf`. Build with `g++ -std=c++17 -O2`. The package contains the files below, and Section 6 explains how to run it.

File	Purpose
<code>bptree.h</code> , <code>bptree.cpp</code>	Your solution (stubs provided)
<code>disk.h</code> , <code>legacy.h</code> , <code>harness.cpp</code>	Provided. Do not modify.
<code>grader/</code>	Python package that simulates the devices, runs the two phases, checks answers and reports costs (Section 6)
<code>cases/</code>	Public test cases

The grader prints, for each case, the correctness verdict, the four counts, and the two phase costs, so that you can compare designs quickly.

9 Grading Rubric

Your code will be graded against the cost incurred by the instructor's reference implementation. The smaller the cost with respect to the reference the better.